CENTRO UNIVERSITÁRIO EUROAMERICANO - UNIEURO

Projeto Simbionte – Documento de Pesquisa

Guilherme Ribeiro
Marcelo Vaz
Sebastião Brayan

Brasília - DF
2026

1. Aplicações Web

Uma aplicação web é um software executado em servidores remotos que é acessado diretamente por navegadores, como Google Chrome, Firefox ou Safari, sem necessidade de instalação local. Esse modelo funciona através da arquitetura cliente-servidor, na qual o cliente realiza requisições ao servidor, que processa as informações e retorna uma resposta apropriada. Em aplicações modernas, o frontend é responsável pela interface visual e interação com o usuário, enquanto o backend gerência regras de negócio, autenticação, processamento de dados e integração com serviços externos. Resumidamente, aplicações web são sistemas hospedados em servidores ou plataformas em nuvem, como Vercel, Render ou AWS, utilizados diariamente por múltiplos usuários através da internet.
Fluxo básico:

```
Usuário → Frontend → Backend → Banco de Dados
```

2. Banco de Dados

O banco de dados é o componente responsável por armazenar, organizar e recuperar informações da aplicação. Ele funciona como a camada de persistência do sistema, garantindo que dados possam ser consultados, modificados e reutilizados posteriormente. Em aplicações web modernas, bancos de dados armazenam:

- usuários
- publicações
- logs
- métricas
- sessões
- informações de contexto
- memória dos agentes

Os bancos de dados mais utilizados atualmente são PostgreSQL, MySQL, MongoDB e Redis.

3. APIs

Uma API (Application Programming Interface) é responsável pela comunicação entre diferentes sistemas. Em aplicações web, a API recebe requisições do frontend, processa as informações necessárias e retorna respostas estruturadas, normalmente em formato JSON.
Exemplo de fluxo:

```
Usuário → Frontend → API → Banco de Dados → API → Frontend
```

A API atua como intermediária entre a interface visual e os serviços internos da aplicação.

4. Orquestração

A orquestração é o processo responsável por coordenar diferentes serviços, APIs, bancos de dados e sistemas internos para que trabalhem de forma integrada e automatizada. Em aplicações modernas, o orquestrador controla:

- fluxo de execução
- comunicação entre serviços
- ordem das operações
- gerenciamento de contexto
- controle de memória
- tratamento de erros
- processamento assíncrono

5. Padrões de Projeto sem LLMs

5.1 MVC (Model-View-Controller)

O padrão MVC é uma das arquiteturas mais utilizadas no desenvolvimento web tradicional. Ele divide a aplicação em três partes:

- Model → responsável pelos dados e regras de negócio
- View → responsável pela interface visual
- Controller → responsável pelo fluxo da aplicação e comunicação entre as camadas

Nesse modelo, o Controller atua como coordenador principal do sistema.

6. Modelos de Linguagem de Grande Escala (LLMs)

LLMs (Large Language Models) são sistemas de Inteligência Artificial projetados para compreender, processar e gerar linguagem natural. Esses modelos são a base de ferramentas modernas de IA generativa, como ChatGPT, Gemini, Claude e Llama. Os LLMs são treinados utilizando grandes volumes de texto coletados da internet, livros, artigos e outras fontes de dados, através de técnicas de Machine Learning e Deep Learning baseadas na arquitetura Transformer. Esses modelos conseguem:

- gerar texto
- resumir informações

- responder perguntas
- interpretar contexto
- criar conteúdo
- tomar decisões em linguagem natural

7. Fine-Tuning

Fine-Tuning é o processo de especialização de um modelo pré-treinado utilizando um conjunto de dados menor e mais específico. Esse processo permite adaptar o modelo para áreas específicas, como:

- saúde
- direito
- programação
- análise financeira
- redes sociais

8. RAG (Retrieval-Augmented Generation)

RAG é uma técnica que combina modelos de linguagem com recuperação de informações externas. Em vez de modificar o conhecimento interno do modelo, o sistema busca informações em:

- documentos
- bancos de dados
- APIs
- arquivos
- bases corporativas

Essas informações são adicionadas ao contexto do prompt antes da geração da resposta. O objetivo do RAG é:

- reduzir alucinações
- aumentar precisão
- fornecer contexto atualizado
- melhorar memória contextual

9. Prompt Engineering

Prompt Engineering é a prática de criar instruções estruturadas para orientar o comportamento do modelo de linguagem. Essas instruções definem:

- personalidade
- formato da resposta
- objetivo
- contexto
- restrições
- comportamento esperado

10. Tipos de LLMs

Os modelos de linguagem podem ser divididos em diferentes categorias:

10.1 Modelos Multimodais

Processam múltiplos tipos de mídia:

- texto
- imagem
- áudio
- vídeo

10.2 Modelos Especialistas

Treinados para domínios específicos:

- medicina
- direito
- programação
- pesquisa científica

10.3 Modelos de Propósito Geral

Projetados para tarefas amplas:

- conversação
- resumo
- geração de conteúdo
- análise textual

11. O que muda com LLMs

Aplicações web tradicionais utilizam apenas frontend, backend e banco de dados. Com a introdução de LLMs, surge uma nova camada na arquitetura responsável pela integração e

orquestração dos modelos de linguagem. Essa camada pode receber diferentes nomes:

- AI Service
- Orchestrator
- Agent Layer
- LLM Pipeline
- AI Gateway

O objetivo dessa camada é coordenar chamadas aos modelos de IA e integrar múltiplos serviços inteligentes dentro da aplicação.

12. Orquestração de Chamadas para LLMs

A orquestração de chamadas é responsável por controlar múltiplas etapas do sistema de IA. O orquestrador decide:

- qual modelo utilizar
- como montar o contexto
- como armazenar memória
- como evitar respostas incorretas
- como processar múltiplos agentes
- como integrar APIs externas
- como armazenar histórico de conversas

13. Arquiteturas Modernas com LLMs

13.1 Arquitetura Cliente-Servidor em Tempo Real

Componente	Descrição
Funcionamento	A aplicação é dividida em: ● Frontend → interface do usuário ● Backend → lógica da aplicação ● Banco de Dados → persistência ● WebSockets → comunicação em tempo real
Fluxo	`Usuário → Frontend React → WebSocket/API REST Backend Node.js → PostgreSQL + Redis →`

Componente	Descrição
	`Eventos atualizam o frontend em tempo real`
Tecnologias Utilizadas	React, Node.js, Express, Socket.IO, PostgreSQL, Redis
Vantagens	• Fácil implementação • Atualização em tempo real • Boa integração frontend/backend • Excelente para dashboards e monitoramento
Limitações	• Backend pode se tornar gargalo • Escalabilidade limitada em grandes volumes • Alto consumo de recursos em múltiplas conexões simultâneas
Referência	https://socket.io/docs/v4/

13.2 Arquitetura de Orquestração de LLMs

Componente	Descrição
Funcionamento	O sistema utiliza uma camada intermediária chamada LLM Gateway responsável por abstrair e padronizar chamadas para diferentes modelos de linguagem
Fluxo	`Agente → Orquestrador → LLM Gateway → OpenAI / Gemini / Claude / Llama → Resposta padronizada`

Componente	Descrição
Tecnologias Utilizadas	LangChain, LlamaIndex, OpenAI API, Google AI Studio, Anthropic API, Ollama
Vantagens	• Facilita troca de modelos • Centraliza autenticação • Reduz acoplamento • Permite métricas e monitoramento
Limitações	• Maior complexidade arquitetural • Possível aumento de latência • Necessidade de controle de custos e rate limits
Referência	https://docs.langchain.com/oss/python/langchain/overview

13.3 Arquitetura Multiagente

Componente	Descrição
Funcionamento	A simulação é executada em ciclos discretos. Cada agente executa: • percepção • decisão • ação • resolução • persistência
Fluxo	`Estado global → Percepção dos agentes → LLM toma decisão → Ação estruturada em JSON → Sistema resolve conflitos → Banco atualiza estado → Novo ciclo`

Componente	Descrição
Tecnologias Utilizadas	Node.js, PostgreSQL, Redis Queue, JSON Schema Validation, Workers Assíncronos
Vantagens	• Excelente para simulações • Fácil coleta de métricas • Escalabilidade baseada em ciclos • Adequado para pesquisa acadêmica
Limitações	• Alto custo computacional • Chamadas paralelas para LLMs podem ser caras • Sincronização complexa em larga escala
Referência	https://arxiv.org/abs/2304.03442

13.4 Arquitetura Event-Driven

Componente	Descrição
Funcionamento	Nesse modelo, todas as ações do sistema são tratadas como eventos. Exemplos: • publicação criada • TrustScore atualizado • verificação realizada • conteúdo viralizado
Fluxo	`Evento acontece → Message Broker → Serviços recebem o evento → Sistema é atualizado ↓ Frontend recebe atualização`
Tecnologias Utilizadas	Apache Kafka, RabbitMQ, Redis Streams

Componente	Descrição
Vantagens	• Alta escalabilidade • Processamento assíncrono • Excelente desacoplamento • Muito utilizado em grandes plataformas
Limitações	• Arquitetura complexa • Debug mais difícil • Consistência eventual dos dados
Referência	https://kafka.apache.org/documentation/

13.5 Arquitetura RAG com Memória Vetorial

Componente	Descrição
Funcionamento	Os agentes possuem memória contextual persistente baseada em embeddings vetoriais. O sistema recupera informações relevantes do histórico antes de enviar o contexto ao modelo de linguagem
Fluxo	`Agente recebe contexto → Sistema busca memória vetorial → Informações relevantes retornam → Prompt é enriquecido → LLM toma decisão`
Tecnologias Utilizadas	pgvector, ChromaDB, Weaviate, OpenAI Embeddings
Vantagens	• Memória contextual • Respostas mais precisas • Simulações mais realistas

Componente	Descrição
	• Melhor comportamento emergente
Limitações	• Maior custo computacional • Complexidade adicional • Necessidade de embeddings e banco vetorial
Referência	https://arxiv.org/abs/2005.11401